# Lógica de Programação Algorítmica – aula 01

ANDRÉ TIBA (andre.tiba@sereducacional.com)

# Sumário

- aula 01: introdução
- aula 02: condicionais
- aula 03: laços
- aula 04: vetores / matrizes
- aula 05: matrizes / funções
- aula 06: funções / revisão

# Sumário aula 1

1) Motivação: por que precisamos programar?
2) Algoritmo
3) O que pensar, ou como pensar na hora de programar?
4) Formas de visualizar um algoritmo/código
5) IDE: Compilação/Execução/Depuração (debbuging)
6) Tipos de variáveis
7) Operador atribuição (=)
8) Operadores Aritméticos
9) Operadores de Comparação

# Materiais do curso

**Pasta do drive com códigos apresentados na aula:**

https://drive.google.com/drive/folders/1YZQb2mVoT5ACQGivNn3dxHrQf5pwnqEl?usp=sharing

# 1) Por que precisamos programar?

- Nosso cérebro é uma ferramenta fantástica, capaz de resolver vários tipos de problemas diferentes, com variados tipos de complexidade.

- Porém temos dificuldade de resolver tarefas repetitivas → começamos a errar, porque nos cansamos.

- Então, usamos o nosso cérebro e criamos uma máquina capaz de realizar as tarefas repetitivas, de maneira que ela faça nosso trabalho (sem erros).

  - Passamos instruções (ordens) à máquina, que ao executar tais instruções, está resolvendo o problema.

# 1) Por que precisamos programar?

- Quando resolvemos um problema, nosso cérebro também executa uma sequencia de instruções, só que de forma muito mais complexa, pois todo o conhecimentos que temos são usados no processo de criação de uma resposta.

- Uma máquina não funciona assim. Ela não tem esse poder de processamento que nosso cérebro tem. Ela é muito mais simples e limitada.

- Por isso, precisamos entender suas limitações, e passar instruções que ela consiga realizar.

  - Isso nos força a pensar diferente. **Essa é a essência da lógica de programação**.

# 2) Algoritmo

- Um algoritmo é uma **sequencia de instruções**, que precisa ser realizada em uma **determinada ordem**.
  - Esta sequencia foi "pensada" para ser realizada, em uma dada ordem, por dois motivos:
    - ao final, resultar na solução do problema
    - **poupar** esforço/trabalho/energia/tempo ou qualquer outro parâmetro.

# 2) Algoritmo

- Pare e pense um instante: relembre a sequencia de ações que você faz, desde que acorda até o momento que sai de casa para seus compromissos diários.
  - Você faz esta mesma sequencia (ou quase igual) diariamente. Por que?
  - Provavelmente porque ela é a que te economiza mais tempo. Ou porque ela é feita desta forma, para que você possa fazer outras atividades.
- Um algoritmo para resolver um problema é assim: pensado para realizar a resolução de um problema minimizando algum custo (operações, tempo, etc ...)

# 2) Algoritmo

- A forma de raciocinar usada em lógica de programação induz a resolver o problema de forma otimizada.
- Quanto mais se pratica, quanto mais se raciocina no formato presente na lógica de programação, mais seu cérebro consegue realizar otimizações.
- A lógica de programação é a forma que raciocinamos, de maneira que a resolução de um problema possa ser realizada em um computador.

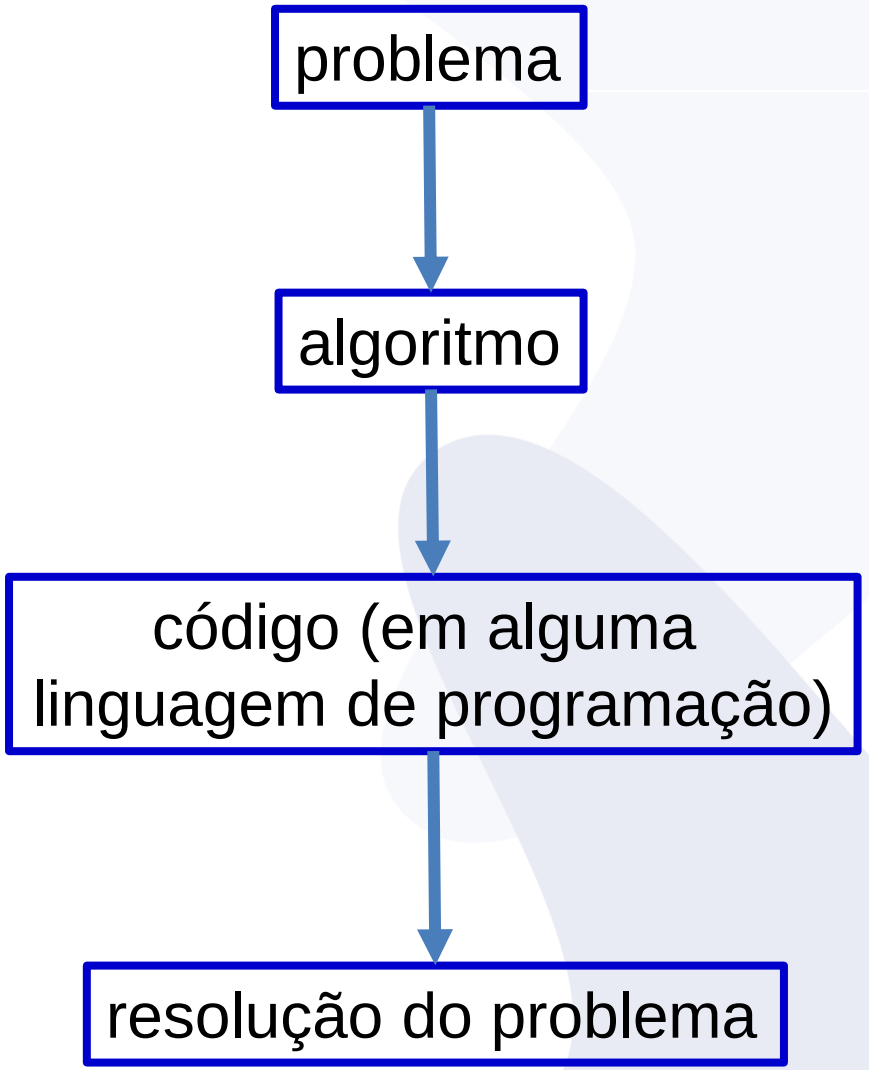
# 2) Algoritmo

- A lógica de programação é a forma que raciocinamos, de maneira que a resolução de um problema possa ser realizada em um computador.

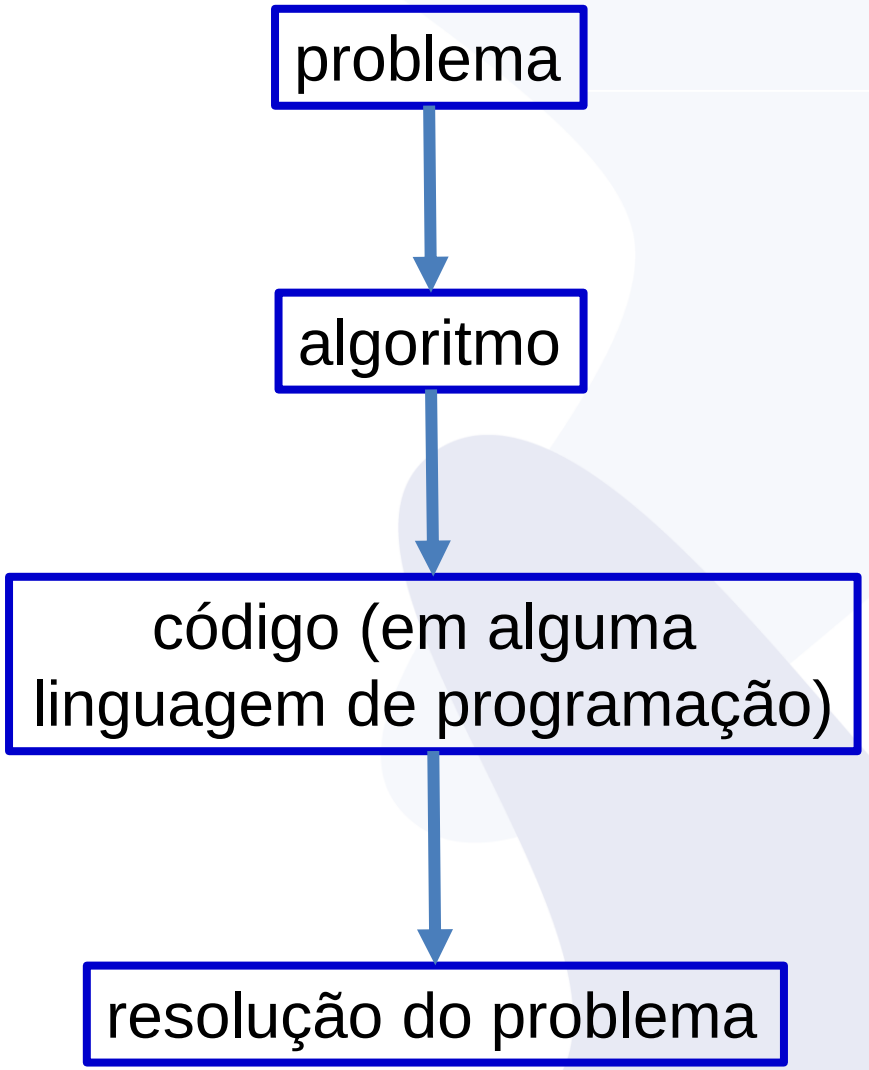
- Para isso, passamos nossas instruções em formato de texto através de uma linguagem interpretável pelo computar (quando codificamos um algoritmo em uma linguagem de programação).

- Esta sequencia de instruções escritas é transformada para um formato binário, que é o formato onde o computador consegue executar as instruções, de fato.

# 2) Algoritmo ou pseudo-código

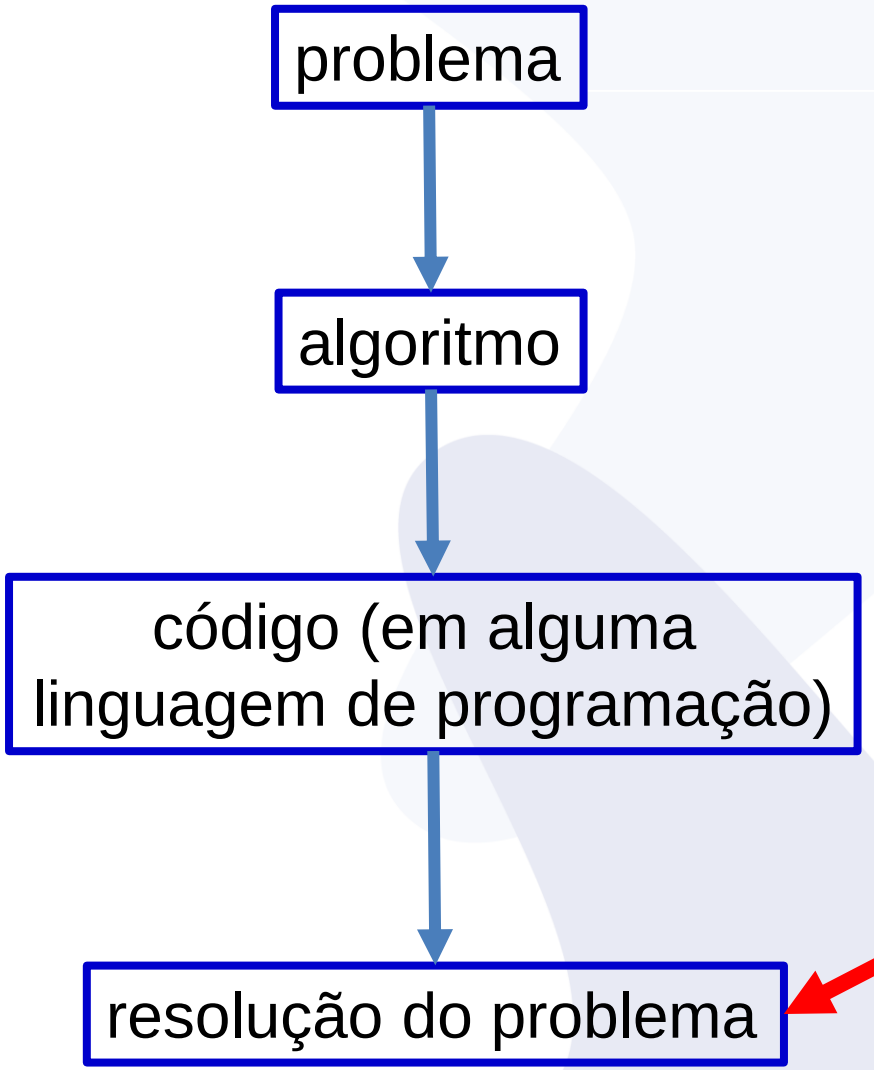


# 2) Algoritmo ou pseudo-código




**Como saber se sua lógica de programação está correta?**

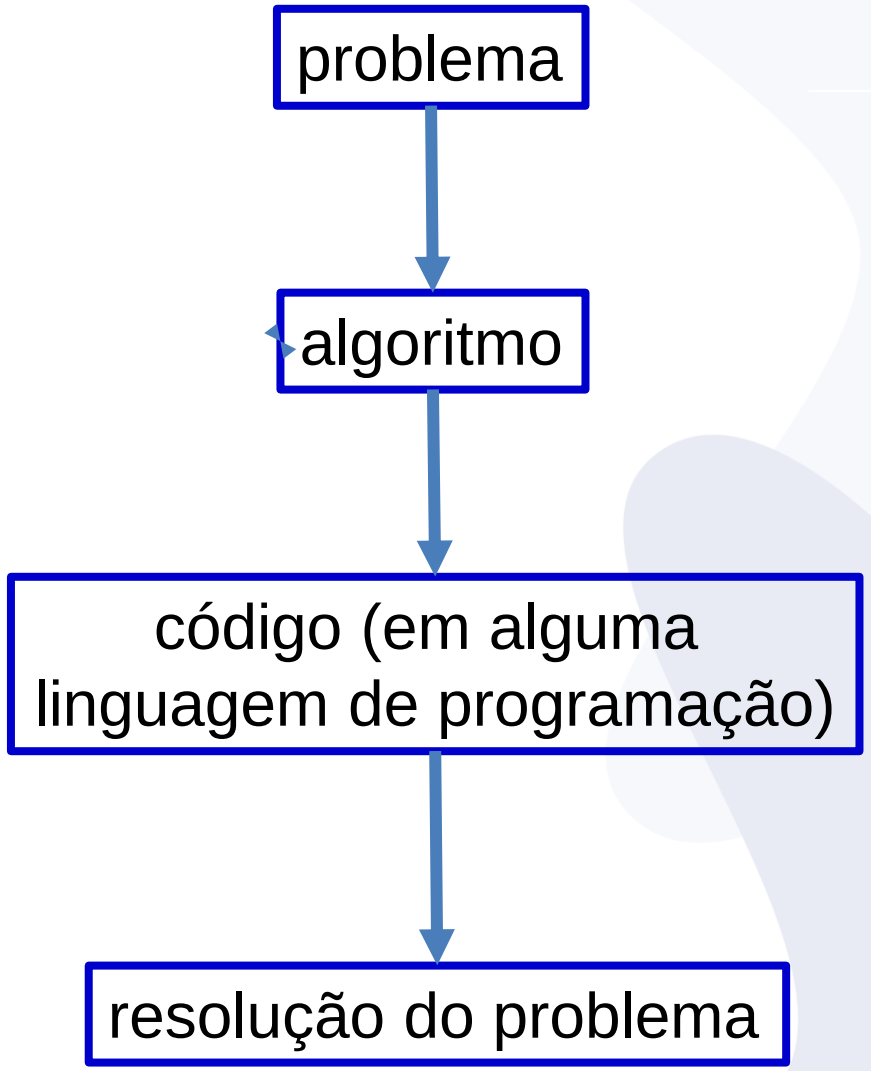
# 2) Algoritmo ou pseudo-código



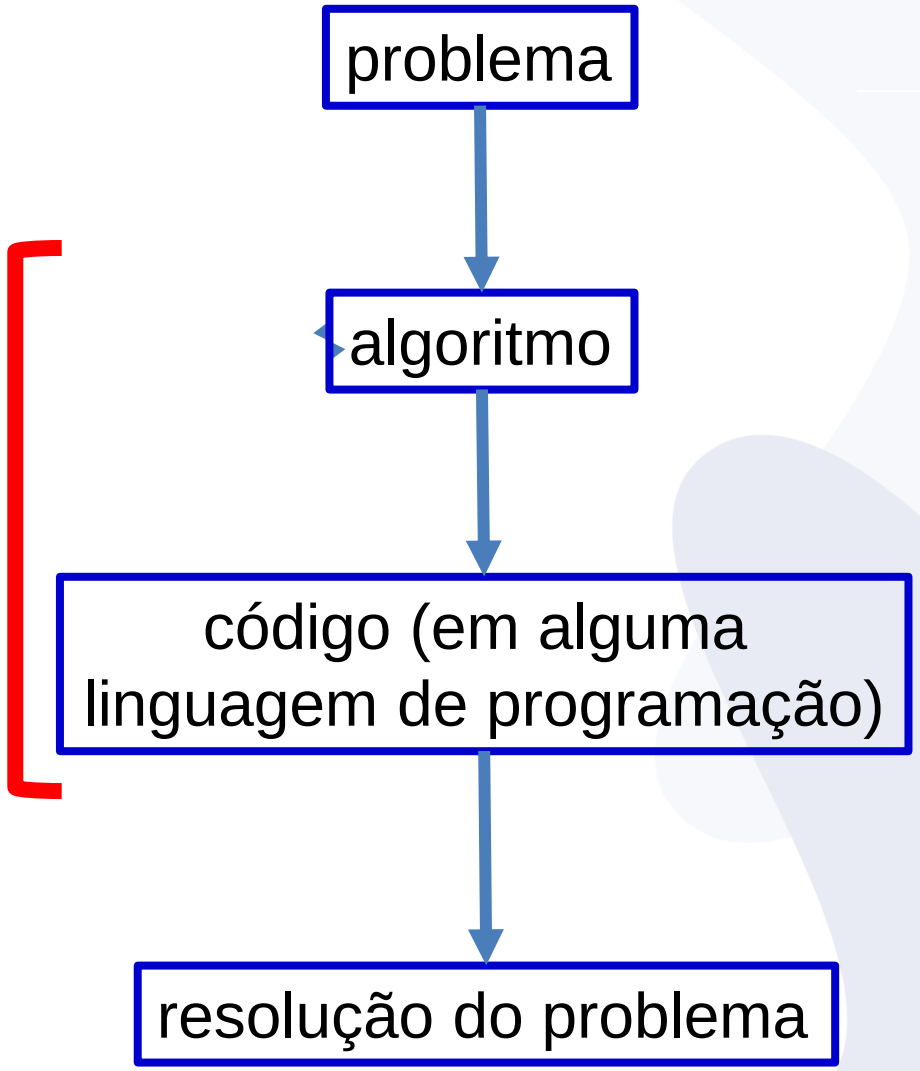

**Como saber se sua lógica de programação está correta?**

**Só terá certeza se implementar um código**

# 3) O que pensar, ou como pensar na hora de programar?

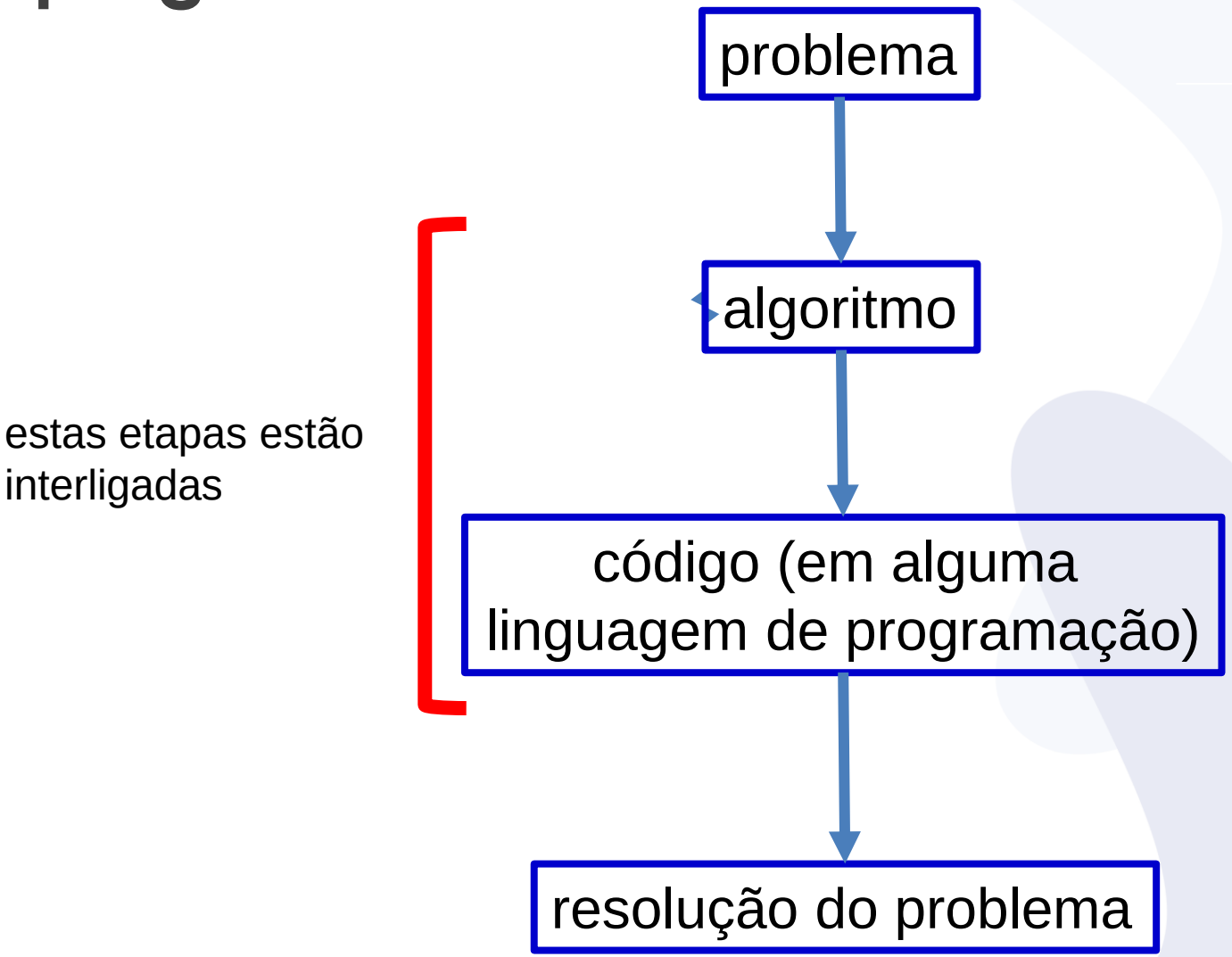


# 3) O que pensar, ou como pensar na hora de programar?




estas etapas estão interligadas

# 3) O que pensar, ou como pensar na hora de programar?




## Três personagens:

**programador**

**programa**

**usuário do programa**

# 3) O que pensar, ou como pensar na hora de programar?

**programador**

**programa**

**usuário do programa é o programador**

# 3) O que pensar, ou como pensar na hora de programar?

programador

programa

usuários do programa

# 3) O que pensar, ou como pensar na hora de programar?

programa e usuário interagem

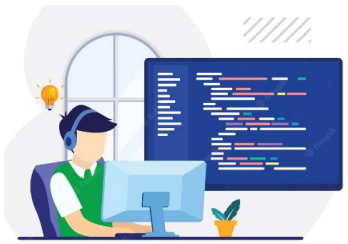
**programador**

**programa**

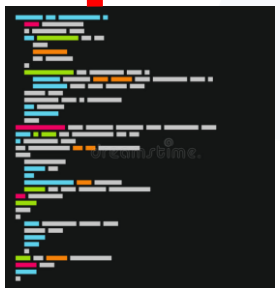
**usuários do programa**

# 3) O que pensar, ou como pensar na hora de programar?

outros programadores podem usar seu código

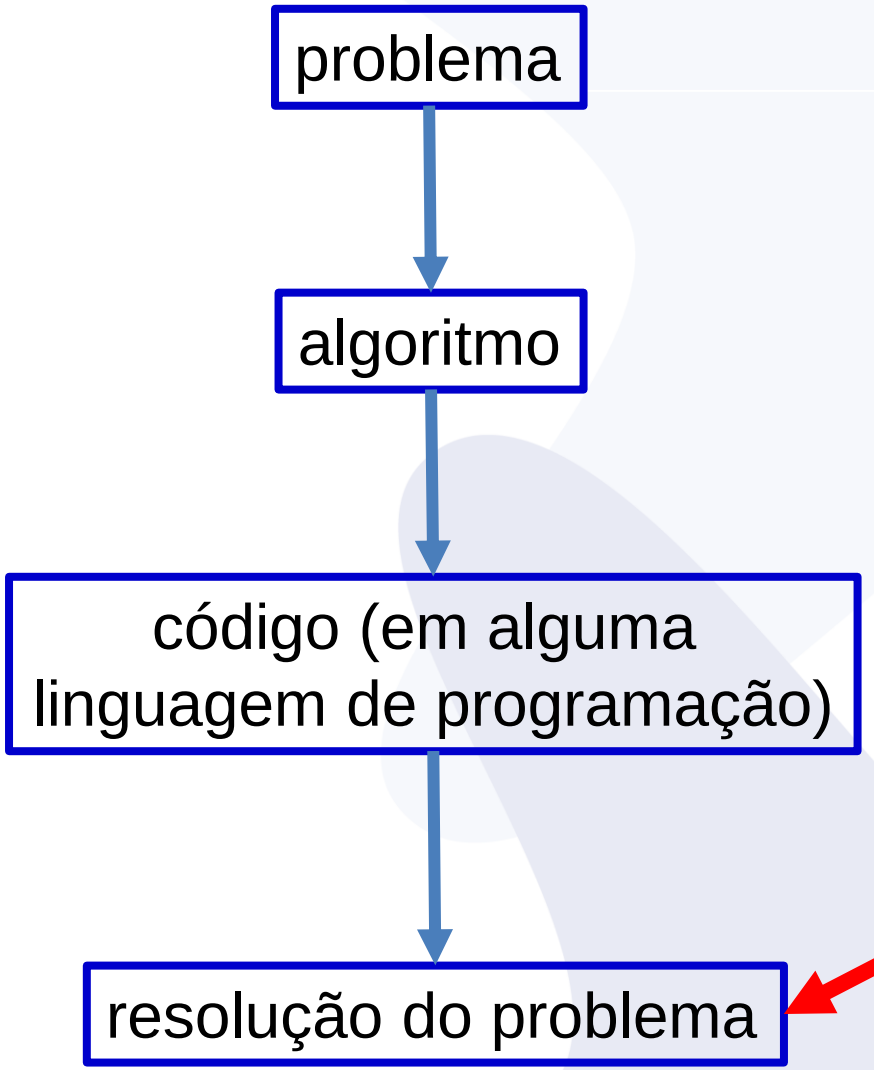


**programadores**



**programa**

# 4) Formas de visualizar Algoritmo/pseudo-código / código




**Como saber se sua lógica de programação está correta?**

**Só terá certeza se implementar um código**

# 4) Formas de visualizar Algoritmo/pseudo-código / código

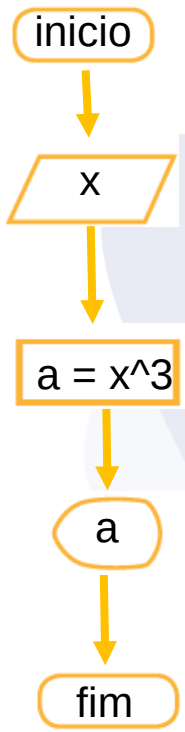
- Exemplo: deseja-se criar um programa que calcule o cubo de um número
  - o programa recebe como entrada um número x
  - o programa imprime na saída o valor de $x^3$.

# 4) Formas de visualizar Algoritmo/pseudo-código / código

**pseudo-código**

```
declaração de variáveis:
    x, a: real
inicio
    leia(x)
    calcule a = x*x*x
    imprima(a)
fim
```

**fluxograma**

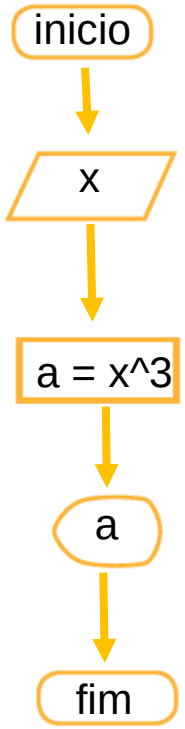


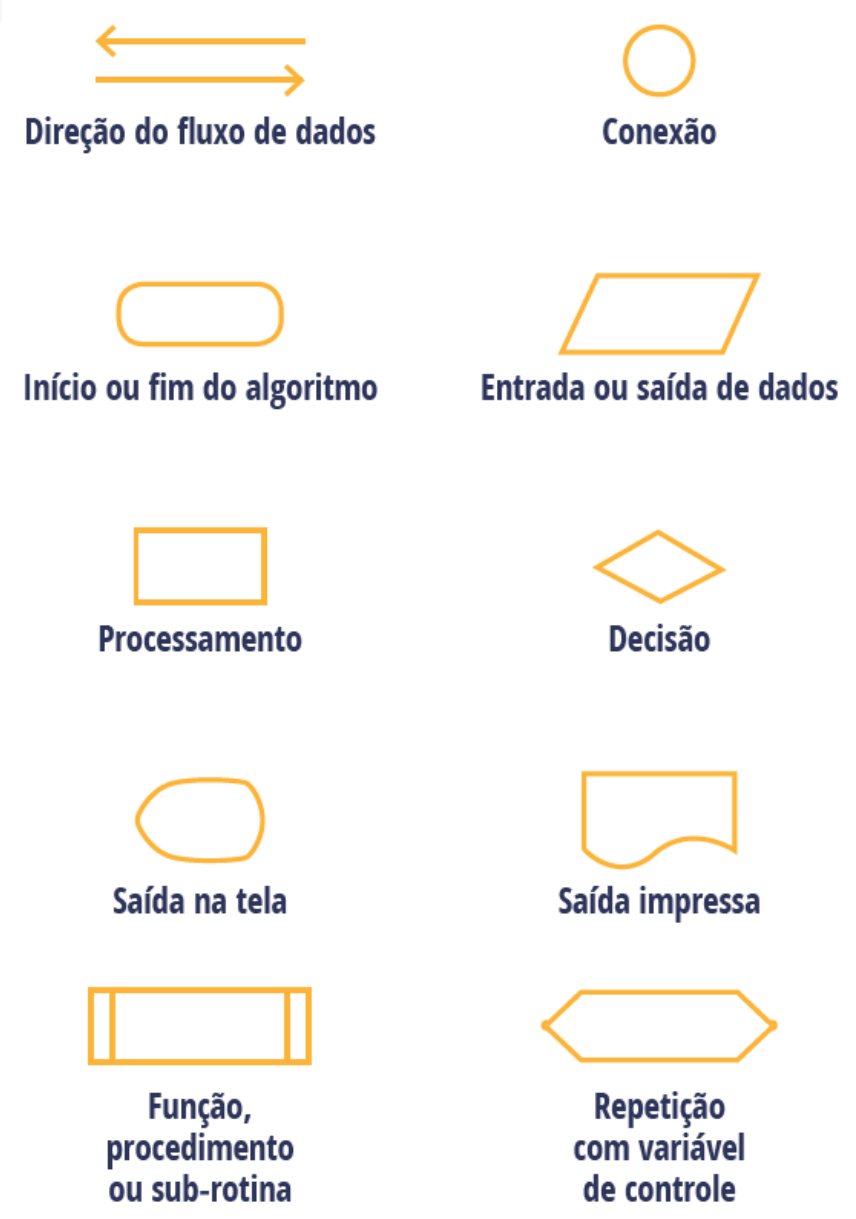

**código (linguagem C)**

```c
/* este programa calcula
  o cubo de um número */
#include <stdio.h>
int main()
{
    float a = 0., x = 0.;
    printf("este programa calcula x^3 \n");
    printf("digite x: ");
    scanf("%f",&x);
    a = x*x*x;
    printf("%f^3 = %f", x, a);
    return 0;
}
```

# 4) Formas de visualizar Algoritmo/pseudo-código / código

**fluxograma**



# 4) Formas de visualizar Algoritmo/pseudo-código / código

**pseudo-código**

declaração de variáveis: } linha **6**
    x, a: real

Inicio } linhas **4 e 5**

    leia(x) } linha **9**
    calcule a = x*x*x } linha **10**
    imprima(a) } linha **11**

fim } linhas **12 e 13**

**código (linguagem C)**

```
1  /* este programa calcula
2    o cubo de um número */
3  #include <stdio.h>
4  int main()
5  {
6      float a = 0., x = 0.;
7      printf("este programa calcula x^3 \n");
8      printf("digite x: ");
9      scanf("%f",&x);
10     a = x*x*x;
11     printf("%f^3 = %f", x, a);
12     return 0;
13 }
```

# 4) Formas de visualizar Algoritmo/pseudo-código / código

**código (linguagem C)**

```c
/* este programa calcula
   o cubo de um número */
#include <stdio.h>
int main()
{
    float a = 0., x = 0.;
    printf("este programa calcula x^3 \n");
    printf("digite x: ");
    scanf("%f",&x);
    a = x*x*x;
    printf("%f^3 = %f", x, a);
    return 0;
}
```

Terminal

```
este programa calcula x^3
digite x:
```

Terminal

```
este programa calcula x^3
digite x: 3
3.000000^3 = 27.000000
```

# 4) Formas de visualizar Algoritmo/pseudo-código / código

```
1  /* este programa calcula
2    o cubo de um número */
3  #include <stdio.h>
4  int main()
5  {
6      float a = 0., x = 0.;
7      printf("este programa calcula x^3 \n");
8      printf("digite x: ");
9      scanf("%f",&x);
10     a = x*x*x;
11     printf("%f^3 = %f", x, a);
12     return 0;
13 }
```

linhas **1** e **2**: comentário, compilador ignora as instruções.

linha **3**: inclusão da biblioteca stdio.h (conjunto pré definido de funções).

linha **4**: todo programa em linguagem C ocorre dentro da função **main(){ }**, sendo delimitada por chaves {}. Na linha **5** a chave **{** inicia as instruções dentro da função main.

linha **6**: declaração e inicialização das variáveis.

linhas **7** e **8**: imprimindo informações no terminal, programa se comunicando com usuário. A função **printf** faz parte da biblioteca stdio.h

linha **9**: lendo informação do teclado com a função **scanf** (também da biblioteca stdio.h) e gravando a informação na variável **x**.

linha **10**: calcula-se o valor de $x^3$ e grava-se o resultado na variável **a**.

linha **11**: impressão do resultado no terminal.

linha **12**: o **return 0** indica a instrução encerramento de instruções da função main().

linha **13**: a chave **}** fecha a função main e o programa foi finalizado.

# 5) IDE: compilação/execução/depuração(debbugin)

- Todo código é realizado dentro de uma IDE (Integrated Development Environment): Ambiente de Desenvolvimento Integrado)

- O IDE é um programa que possui três ambientes necessários para programação

  - (1) editor de texto → as instruções serão escritas pelo programador.

  - (2) compilador da linguagem escolhida → transforma as instruções de texto em instruções em binário, que podem ser executados pelo computador

  - (3) depurador (debbuging) → ferramenta importante para executar o código linha a linha.

# 5) IDE: compilação/execução/depuração(debbugin)

codingground | Online C Compiler

Execute | Beautify | Share | Source Code | Help | Terminal

```c
/* este programa calcula
  o cubo de um número */
#include <stdio.h>
int main()
{
    float a = 0., x = 0.;
    printf("este programa calcula x^3 \n");
    printf("digite x: ");
    scanf("%f",&x);
    a = x*x*x;
    printf("%f^3 = %f", x, a);
    return 0;
}
```

# 5) IDE: compilação/execução/depuração(debbugin)

compilador

codingground | Online C Compiler

Execute | Beautify | Share | Source Code | Help

```c
/* este programa calcula
  o cubo de um número */
#include <stdio.h>
int main()
{
    float a = 0., x = 0.;
    printf("este programa calcula x^3 \n");
    printf("digite x: ");
    scanf("%f",&x);
    a = x*x*x;
    printf("%f^3 = %f", x, a);
    return 0;
}
```

Terminal

editor de texto

terminal: local de interação entre programa e usuário

# 5) IDE: compilação/execução/depuração(debbugin)

codingground | Online C Compiler

Execute | Beautify | Share | Source Code | Help | Terminal

```c
/* este programa calcula
  o cubo de um número */
#include <stdio.h>
int main()
{
    float a = 0., x = 0.;
    printf("este programa calcula x^3 \n");
    printf("digite x: ");
    scanf("%f",&x);
    a = x*x*x;
    printf("%f^3 = %f", x, a);
    return 0;
}
```

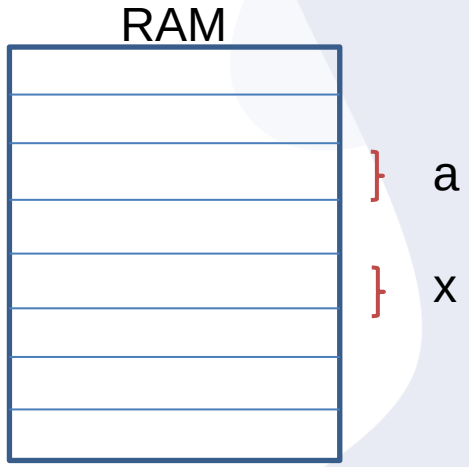
cada IDE possui uma combinação de cores das palavras, de maneira a destacar e facilitar a leitura do código

# 6) Tipos de variáveis

- No problema do cálculo do cubo de um número, era necessário armazenar DUAS informações: o número, e seu valor ao cubo.

- Para tanto, foi necessário declarar DUAS variáveis. A declaração de variáveis é um **pedido de espaço de memória RAM**, para que fique reservado, quando o programa está sendo executado.

declaração de variáveis:
x, a: real




```c
/* este programa calcula
  o cubo de um número */
#include <stdio.h>
int main()
{
    float a = 0., x = 0.;
    printf("este programa calcula x^3 \n");
    printf("digite x: ");
    scanf("%f",&x);
    a = x*x*x;
    printf("%f^3 = %f", x, a);
    return 0;
}
```

# 6) Tipos de variáveis

- Uma variável do tipo **inteiro (int)** demanda de um espaço de memória na RAM **menor** que o espaço de memória demandado por uma variável **real (float / double)**.

- Uma variável do tipo **caractere (string)** precisa de **menos** espaço de memória que uma variável do tipo **inteiro (int)**.

- Uma variável do tipo **lógica**, conhecida também como **booleana (bool)** só pode assumir valor verdadeiro (true), ou falso (false).

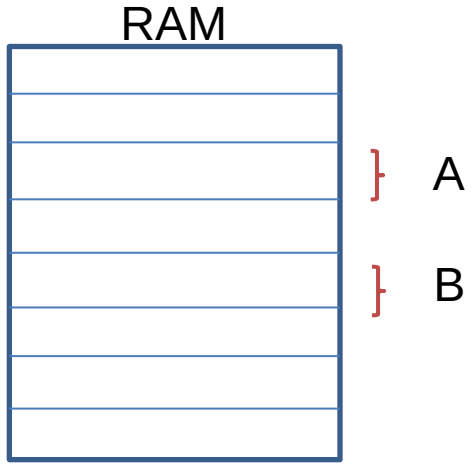
  - Algumas linguagens **não** possuem o tipo booleano, sendo associado o verdadeiro ao inteiro 1, e o falso ao inteiro 0.

# 7) Operador atribuição (=)

- O operador “=” representa atribuição

# 7) Operador atribuição (=)

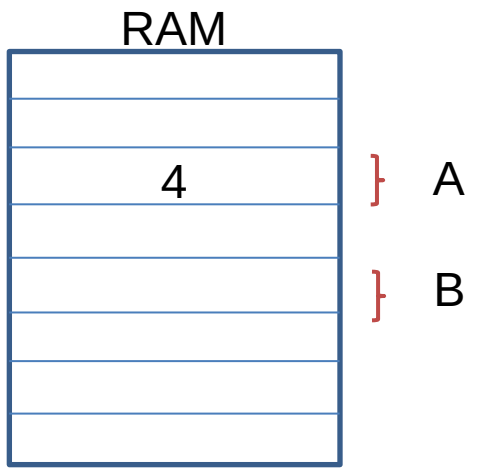
- O operador “=” representa atribuição
- Imagine que as variáveis A e B foram declaradas.



# 7) Operador atribuição (=)

- O operador “=” representa atribuição

A = 4 (lê-se: atribua à variável A, o valor 4).

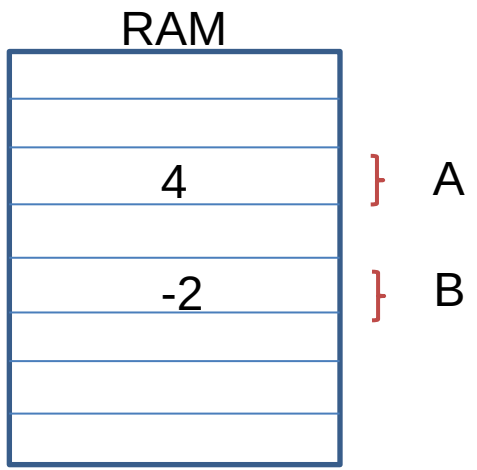


# 7) Operador atribuição (=)

- O operador "=" representa atribuição

A = 4 (lê-se: atribua à variável A, o valor 4).

B = -2 (lê-se: atribua à variável B, o valor -2).



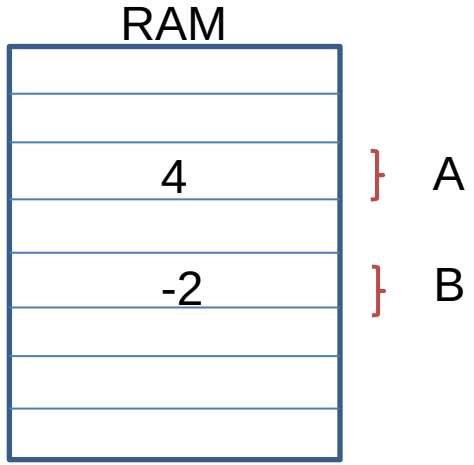
# 7) Operador atribuição (=)

- O operador "=" representa atribuição

A = 4 (lê-se: atribua à variável A o valor 4)

B = -2 (lê-se: atribua à variável B o valor -2)

```c
#include <stdio.h>
int main()
{
    int A = 4, B = -2;
    printf("\n endereço de A: %i",&A);
    printf("   conteudo de A: %i",A);
    printf("\n endereço de B: %i",&B);
    printf("   conteudo de B: %i",B);
    return 0;
}
```




```
endereço de A: 1587485136   conteudo de A: 4
endereço de B: 1587485140   conteudo de B: -2

...Program finished with exit code 0
Press ENTER to exit console.
```

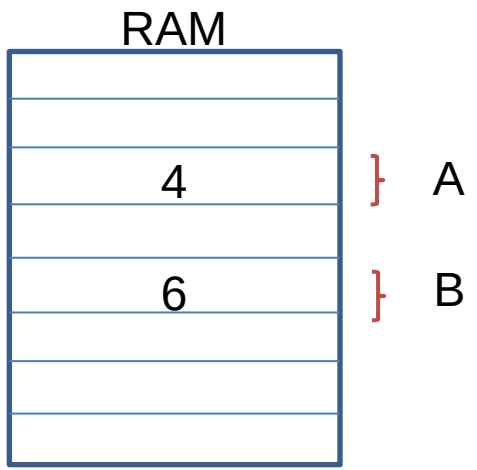
# 7) Operador atribuição (=)

- O operador "=" representa atribuição

A = 4 (lê-se: atribua à variável A, o valor 4).

B = -2 (lê-se: atribua à variável B, o valor -2).

B = A + 2 (lê-se: atribua à variável B, o valor contido em A).



# 7) Operador atribuição (=)

- O operador "=" representa atribuição

A = 4 (lê-se: atribua à variável A,

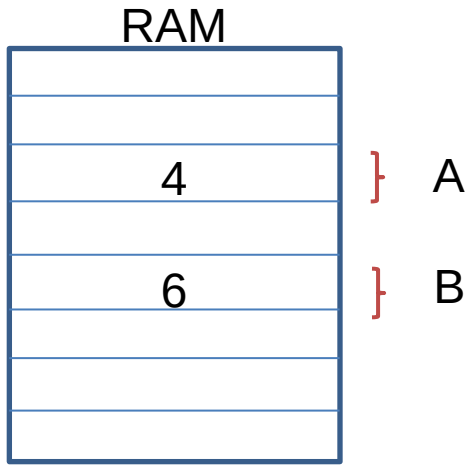
B = -2 (lê-se: atribua à variável B,

B = A + 2 (lê-se: atribua à variáve

```c
#include <stdio.h>
int main()
{
    int A = 4, B = -2;
    B = A + 2;
    printf("\n endereço de A: %i",&A);
    printf("   conteudo de A: %i",A);
    printf("\n endereço de B: %i",&B);
    printf("   conteudo de B: %i",B);
    return 0;
}
```

```
endereço de A: 658982960   conteudo de A: 4
endereço de B: 658982964   conteudo de B: 6

...Program finished with exit code 0
Press ENTER to exit console.
```



# 7) Operador atribuição (=)

- O operador "=" representa atribuição

  4 = A → **<u>não faz sentido, está errado!!!!</u>**

- O lado esquerdo do operador "=" SEMPRE tem uma variável. O lado direto tem ou uma variável, ou um valor (inteiro/real/caractere/lógico), ou mas expressão.

# 8) Operadores aritméticos

| Operador | Descrição |
|---|---|
| + | **Adição**: efetua a soma entre dois números. |
| - | **Subtração**: efetua a subtração entre dois números. |
| * | **Multiplicação**: efetua a multiplicação entre dois números. |
| / | **Divisão decimal**: efetua a divisão entre dois números e dá como resultado um número em ponto flutuante. |
| \ | **Divisão inteira**: efetua a divisão entre dois números inteiros e dá como resultado um número inteiro, desprezando a parte fracionária, caso exista. |
| % | **Resto de divisão**: retorna o resto de uma divisão não exata entre dois números inteiros. |

declaração de variáveis:

A = 1, B = 2, C = 3: inteiro

D: real

Exemplos:

C = 4 + 1, A = A + 2, C = 5 + B + A

A = A - 3, B = 3 – (–2), C = –5 – B + A

A = 3*A – 2, B = A*C, C = –5*B*A +1

D = A /(–2), D = 2/C, D = –5*B/C

A = A \(–2), B = 2\C, D = –5*B\C

B = A%2, C = B%C, A = B%2

# 8) Operadores aritméticos

declaração de variáveis:

A = 1, B = 2, C = 3: inteiro

D: real

```c
// exemplos de operações aritméticas
#include <stdio.h>
int main()
{
    int A = 1, B = 2, C = 3;
    printf(" A + 2: %i\n",A+2);
    printf(" B - 1: %i\n",B - 1);
    printf(" 3*A - 2: %i\n",3*A - 2);
    printf(" A/(-2): %i\n",A/(-2));
    printf(" A/(-2): %f\n",A/(-2.));
    printf(" 6%3: %d\n",6%3);
    printf(" C%2: %d\n",C%2);
    return 0;
}
```

```
A + 2: 3
B - 1: 1
3*A - 2: 1
A/(-2): 0
A/(-2): -0.500000
6%3: 0
C%2: 1


...Program finished with exit code 0
Press ENTER to exit console.
```

# 9) Operadores de comparação

| Operador | Descrição |
|---|---|
| < | Menor que |
| > | Maior que |
| <= | Menor que ou igual a |
| >= | Maior que ou igual a |
| == | Igual a |
| <> != | Diferente de |

declaração de variáveis:
A = 1, B = 2, C = 3, D: inteiro

Exemplos:

A + 1 > 2 → falso (0)

A + 1 >= 2 → verdadeiro (1)

B == 2*A → verdadeiro (1)

A != C/3 → falso (0)

D = A - 1 > 0 → o teste é falso, portanto 0 é armazenado na variável D

D = 3*A - B < 2 → o teste é verdadeiro, portanto 1 é armazenado na variável D

# 9) Operadores de comparação

declaração de variáveis:

A = 1, B = 2, C = 3, D: inteiro

Run Debug Stop Share Save { } Beautify

main.c

```c
// exemplos de operações de comapração
// a resposta é um teste lógico
#include <stdio.h>
int main()
{
    int A = 1, B = 2, C = 3, D;
    printf(" A < 2: %i\n",A < 2);
    printf(" B - A < 0: %i\n",B - A < 0);
    printf(" 2*B - 4*A >= 0: %i\n",2*B - 4*A >= 0);
    printf(" 3*B == 2*C: %i\n",3*B == 2*C);
    printf(" C != 1 + A: %i\n",C != 1 + A);
    printf(" 6%3 > 0: %i\n", 6%3 > 0);
    printf(" C%2 <= 1: %i\n",C%2 <= 1);
    D = A > 3;
    printf(" D: %i",D);
    return 0;
}
```

```
A < 2: 1
B - A < 0: 0
2*B - 4*A >= 0: 1
3*B == 2*C: 1
C != 1 + A: 1
6%3 > 0: 0
C%2 <= 1: 1
D: 0

...Program finished with exit code 0
Press ENTER to exit console.
```

# OBRIGADO(A)

ser educacional
gente criando o futuro